

Coordination & Execution

When you type `wg service start --max-agents 5`, a background process wakes up, binds a Unix socket, and begins to breathe. Every few seconds it opens the graph file, scans for ready tasks, and decides what to do. This is the coordinator—the scheduling brain that turns a static directed graph into a running system. Without it, workgraph is a notebook. With it, workgraph is a machine.

This section walks through the full lifecycle of work: from the moment the daemon starts, through the dispatch of agents, to the handling of their success, failure, and unexpected death.

The Service Daemon

The service daemon is a background process that hosts the coordinator, listens on a Unix socket for commands, and manages agent lifecycle. It is started with `wg service start` and stopped with `wg service stop`. Between those two moments it runs a loop: accept connections, process IPC requests, and periodically run the coordinator tick.

The daemon writes its PID and socket path to `.workgraph/service/state.json`—a lockfile of sorts. When you run `wg service status`, the CLI reads this file, checks whether the PID is alive, and reports the result. If the daemon crashes and leaves a stale state file, the next `wg service start` detects the dead PID, cleans up, and starts fresh. If you want to be forceful about it, `wg service start --force` kills any existing daemon before launching a new one.

All daemon activity is logged to `.workgraph/service/daemon.log`, a timestamped file with automatic rotation at 10 MB. The log captures every coordinator tick, every spawn, every dead agent detection, every IPC request. When something goes wrong, the answer is almost always in this file.

One detail matters more than it might seem: agents spawned by the daemon are *detached*. The spawn code calls `setsid()` to place each agent in its own session and process group. This means agents survive daemon restarts. You can stop the daemon, reconfigure it, start it again, and every running agent continues undisturbed. The daemon does not own its agents—it launches them and watches them from a distance.

The Coordinator Tick

The coordinator’s heartbeat is the *tick*—a single pass through the scheduling logic. Two things trigger ticks: IPC events (immediate, reactive) and a background poll timer (a safety net that catches manual edits to the graph file). The poll interval defaults to 60 seconds and is configurable via `config.toml` or `wg service reload --poll-interval N`.

Each tick has six phases:

1. **Reap zombies.** Even though agents run in their own sessions, they remain children of the daemon process. When an agent exits, it becomes a zombie until the parent calls `waitpid`. The tick begins by reaping all zombies so that subsequent PID checks return accurate results.
2. **Clean up dead agents and count slots.** The coordinator walks the agent registry and checks each alive agent’s PID. If the process is gone, the agent is dead. Dead agents have their tasks unclaimed—the task status reverts to open, ready for re-dispatch. The coordinator then counts truly alive agents (not just registry entries, but processes with running PIDs) and compares against `max_agents`. If all slots are full, the tick ends early.
3. **Build auto-assign meta-tasks.** If `auto_assign` is enabled in the agency configuration, the coordinator scans for ready tasks that have no agent identity bound to them. For each, it creates an `assign-{task-id}` meta-task that the original task is after. This meta-task, when dispatched, will spawn an assigner agent that inspects the agency’s roster and picks the best fit. The meta-

task is tagged "assignment" to prevent recursive auto-assignment—the coordinator never creates an assignment task for an assignment task.

4. **Build auto-evaluate meta-tasks.** If `auto_evaluate` is enabled, the coordinator creates `evaluate-{task-id}` meta-tasks that are after each work task. When the work task reaches a terminal status, the evaluation task becomes ready. Evaluation tasks use the shell executor to run `wg evaluate`, which spawns a separate evaluator to score the work. Tasks assigned to human agents are skipped—the system does not presume to evaluate human judgment. Meta-tasks tagged "evaluation", "assignment", or "evolution" are excluded to prevent infinite regress.
5. **Save graph and find ready tasks.** If the auto-assign or auto-evaluate phases modified the graph (adding meta-tasks, adjusting dependencies), the coordinator saves it before proceeding. Then it computes the set of ready tasks. If no tasks are ready, the tick ends. If all tasks in the graph are terminal, the coordinator logs that the project is complete.
6. **Spawn agents.** For each ready task, up to the number of available slots, the coordinator dispatches an agent. This is where the dispatch cycle—the core of the system—begins.

```
TICK LOOP

1. reap_zombies()
2. cleanup_dead_agents → count alive slots
3. build_auto_assign_tasks (if enabled)
4. build_auto_evaluate_tasks (if enabled)
5. save graph → find ready tasks
6. spawn_agents_for_ready_tasks(slots_available)

Triggered by: IPC graph_changed | poll timer
```

Listing 1: The six phases of a coordinator tick.

The Dispatch Cycle

Dispatch is the act of selecting a ready task and spawning an agent for it. It is not a single operation but a sequence with careful ordering, because the coordinator must prevent double-dispatch: two ticks must never spawn two agents on the same task.

For each ready task, the coordinator proceeds as follows:

Resolve the executor. If the task has an `exec` field (a shell command), the executor is `shell`—no AI agent needed. Otherwise, the coordinator checks whether the task has an assigned agent identity. If it does, it looks up that agent's `executor` field (which might be `claude`, `shell`, or a custom executor). If no agent is assigned, the coordinator falls back to the service-level default executor (typically `claude`).

Resolve the model. Model selection follows a priority chain: the task's own `model` field takes precedence, then the coordinator's configured model, then the agent identity's model preference. This lets you pin specific tasks to specific models—a cheap model for routine evaluation tasks, a capable one for complex implementation.

Build context from dependencies. The coordinator reads each terminal dependency's artifacts (file paths recorded by the previous agent) and recent log entries. This context is injected into the prompt so the new agent knows what upstream work produced and what decisions were made. The agent does not start from a blank slate—it inherits the trail of work that came before it.

Render the prompt. The executor's prompt template is filled with template variables:

`{{task_id}}, {{task_title}}, {{task_description}}, {{task_context}}, {{task_identity}}`.

The identity block—the agent's role, motivation, skills, and operational parameters—comes from resolving the assigned agent's role and motivation from agency storage. Skills are resolved at this point: file skills read from disk, URL skills fetch via HTTP, inline skills expand in place. The rendered prompt is written to a file in the agent's output directory.

For tasks that are part of a structural cycle, the rendered prompt carries additional context: the current `loop_iteration` (which pass this is) and a note about the `--converged` flag. This informs the agent that it can signal `wg done <task-id> --converged` to stop the cycle early—preventing further iteration even if `max_iterations` hasn't been reached and guard conditions are met. The "converged" tag is placed on the cycle header regardless of which member the agent completes. The cycle evaluator checks for this tag before re-opening members for the next iteration. This mechanism exists because cycles that run to `max_iterations` when the work has already stabilized waste compute and agent time. Convergence is the agent's way of saying "the work is stable, no more iterations needed." A subsequent `wg retry` clears the convergence tag, allowing the cycle to resume.

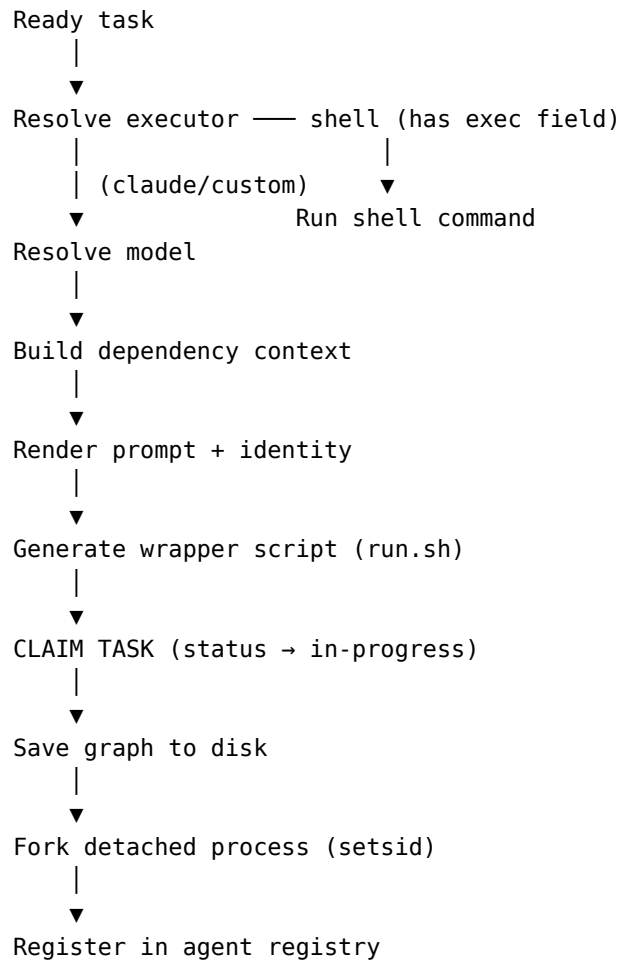
Generate the wrapper script. The coordinator writes a `run.sh` that:

- Unsets `CLAUDECODE` and `CLAUDE_CODE_ENTRYPOINT` environment variables so the spawned agent starts a clean session.
- Pipes the prompt file into the executor command (e.g., `cat prompt.txt | claude --print --verbose --output-format stream-json`).
- Captures all output to `output.log`.
- After the executor exits, checks whether the task is still in-progress. If the agent already called `wg done` or `wg fail`, the wrapper does nothing. If the task is still in-progress and the executor exited cleanly, the wrapper calls `wg done`. If it exited with an error, the wrapper calls `wg fail`. This safety net ensures tasks never get stuck in-progress after an agent dies silently.

Claim the task. Before spawning the process, the coordinator atomically sets the task's status to in-progress and records the agent ID in the `assigned` field. The graph is saved to disk at this point. If two coordinators somehow ran simultaneously, the second would find the task already claimed and skip it. The ordering is deliberate: claim first, spawn second. If the spawn fails, the coordinator rolls back the claim—reopening the task so it can be dispatched again.

Fork the detached process. The wrapper script is launched via `bash run.sh` with `stdin`, `stdout`, and `stderr` redirected. The `setsid()` call places the agent in its own session. The coordinator records the PID in the agent registry.

Register in the agent registry. The agent registry (`.workgraph/agents/registry.json`) tracks every spawned agent: ID, PID, task, executor, start time, heartbeat, status. The coordinator uses this registry to monitor agents across ticks.



Listing 2: The dispatch cycle, from ready task to running agent.

The Wrapper Script

The wrapper script deserves its own discussion because it solves a subtle problem: what happens when an agent dies without reporting its status?

An agent is expected to call `wg done <task-id>` when it finishes or `wg fail <task-id> --reason "..."` when it cannot complete the work. But agents crash. They get OOM-killed. Their SSH connections drop. The Claude CLI segfaults. In all these cases, the task would remain in-progress forever without the wrapper.

The wrapper runs the executor command, captures its exit code, then checks the task's current status via `wg show`. If the task is still in-progress—meaning the agent never called `wg done` or `wg fail`—the wrapper steps in. A clean exit (code 0) triggers `wg done`; a non-zero exit triggers `wg fail` with the exit code as the reason.

This two-layer design (agent self-reports, wrapper as fallback) means the system tolerates both well-behaved and badly-behaved agents. A good agent calls `wg done` partway through the wrapper execution, and when the wrapper later checks, it finds the task already done and does nothing. A crashing agent leaves the task in-progress, and the wrapper picks up the pieces.

Parallelism Control

The `max_agents` parameter is the single throttle on concurrency. When you start the service with `--max-agents 5`, the coordinator will never have more than five agents running simultaneously. Each tick counts truly alive agents (verifying PIDs, not just trusting the registry) and only spawns into available slots.

This is a global cap, not per-task. Five agents might all be working on independent tasks in a fan-out pattern, or they might be serialized through a linear chain with only one active at a time. The coordinator does not reason about the graph's topology when deciding how many agents to spawn—it simply fills available slots with ready tasks, first-come-first-served.

You can change `max_agents` without restarting the daemon. `wg service reload --max-agents 10` sends a Reconfigure IPC message; the coordinator picks up the new value on the next tick. This lets you scale up when a fan-out creates many parallel tasks, then scale back down when work converges.

Map/Reduce Patterns

Parallelism in workgraph arises naturally from the graph structure. A *fan-out* (map) pattern occurs when one task is before several children: the parent completes, all children become ready simultaneously, and the coordinator spawns agents for each (up to `max_agents`). A *fan-in* (reduce) pattern occurs when several tasks are before a single aggregator: the aggregator only becomes ready when all its predecessors are terminal, and then a single agent handles the synthesis.

These patterns are not built-in primitives. They emerge from dependency edges. A project plan that says “write five sections, then compile the manual” naturally produces a fan-out of five writer tasks followed by a fan-in to a compiler task. The coordinator handles this without any special configuration—`max_agents` determines how many of the five writers run concurrently.

Auto-Assign

When the agency system is active and `auto_assign` is enabled in configuration, the coordinator automates the binding of agent identities to tasks. Without auto-assign, a human must run `wg assign <task-id> <agent-hash>` for each task. With it, the coordinator handles matching.

The mechanism is indirect. The coordinator does not contain matching logic itself. Instead, it creates a blocking `assign-{task-id}` meta-task for each unassigned ready task. This meta-task is dispatched like any other—an assigner agent (itself an agency entity with its own role and motivation) is spawned to evaluate the available agents and pick the best fit. The assigner reads the agency roster via `wg agent list`, compares capabilities to task requirements, considers performance history, and calls `wg assign <task-id> <agent-hash>` followed by `wg done assign-{task-id}`.

The result is a two-phase dispatch: first the assigner runs, binding an identity to the task. The assignment task completes, unblocking the original task. On the next tick, the original task is ready again—now with an agent identity attached—and the coordinator dispatches it normally.

Meta-tasks tagged “assignment”, “evaluation”, or “evolution” are excluded from auto-assignment. This prevents the coordinator from creating an assignment task for an assignment task, which would recurse infinitely.

Auto-Evaluate

When `auto_evaluate` is enabled, the coordinator creates evaluation meta-tasks for completed work. For every non-meta-task in the graph, an `evaluate-{task-id}` task is created that is after the original. When the original task reaches a terminal status (done or failed), the evaluation task becomes ready and is dispatched.

Evaluation tasks use the shell executor to run `wg evaluate <task-id>`, which spawns a separate evaluator that reads the task definition, artifacts, and output logs, then scores the work on four dimensions: correctness (40% weight), completeness (30%), efficiency (15%), and style adherence (15%). The scores propagate to the agent, its role, and its motivation, building the performance data that drives evolution (see §5).

Two exclusions apply. Tasks assigned to human agents are not auto-evaluated—the system does not presume to score human work. And tasks that are themselves meta-tasks (tagged "evaluation", "assignment", or "evolution") are excluded to prevent evaluation of evaluations.

Failed tasks also get evaluated. When a task's status is failed, the coordinator removes the predecessor from the evaluation task so it becomes ready immediately. This is deliberate: failure modes carry signal. An agent that fails consistently on certain kinds of tasks reveals information about its role-motivation pairing that the evolution system can act on.

Evaluations created by auto-evaluate carry a source field set to "llm", identifying them as internal assessments from the LLM evaluator. External evaluations can be recorded via `wg evaluate record --task <id> --source <tag> --score <0.0-1.0>`, where the source tag is a freeform string—"outcome:sharpe", "ci:test-suite", "vx:peer-123", or any label meaningful to the project. The evolver reads all evaluations regardless of source (see §5), enabling it to weigh internal quality assessments against external outcome data when proposing improvements to the agency.

Dead Agent Detection and Triage

Every tick, the coordinator checks whether each agent's process is still alive. A dead agent—one whose PID no longer exists—triggers cleanup: the agent's task is unclaimed (status reverts to open), and the agent is marked dead in the registry.

But simple restart is wasteful when the agent made significant progress before dying. This is where *triage* comes in.

When `auto_triage` is enabled in the agency configuration, the coordinator does not immediately unclaim a dead agent's task. Instead, it reads the agent's output log and sends it to a fast, cheap LLM (defaulting to Haiku) with a structured prompt. The triage model classifies the result into one of three verdicts:

- **Done.** The work appears complete—the agent just didn't call `wg done` before dying. The task is marked done, and cycle iteration is evaluated.
- **Continue.** Significant progress was made. The task is reopened with recovery context injected into its description: a summary of what was accomplished, with instructions to continue from where the previous agent left off rather than starting over.
- **Restart.** Little or no meaningful progress. The task is reopened cleanly for a fresh attempt.

Both "continue" and "restart" respect `max_retries`. If the retry count exceeds the limit, the task is marked failed rather than reopened. The triage model runs synchronously with a configurable timeout (default 30 seconds), so it does not block the coordinator for long.

This three-way classification turns agent death from a binary event (restart or give up) into a nuanced recovery mechanism. A task that was 90% complete when the agent was OOM-killed does not lose its progress.

IPC Protocol

The daemon listens on a Unix socket (`.workgraph/service/daemon.sock`) for JSON-line commands. Every CLI command that modifies the graph—`wg add`, `wg done`, `wg fail`, `wg retry`—automatically sends a `graph_changed` message to wake the coordinator for an immediate tick.

The full set of IPC commands:

Command	Effect
<code>graph_changed</code>	Schedules an immediate coordinator tick. The fast path for reactive dispatch.

Command	Effect
spawn	Directly spawns an agent for a specific task, bypassing the coordinator's scheduling.
agents	Returns the list of all registered agents with their status, PID, and uptime.
kill	Terminates a running agent by PID (graceful SIGTERM, then SIGKILL if forced).
status	Returns the coordinator's current state: tick count, agents alive, tasks ready.
shutdown	Stops the daemon. Running agents continue independently by default; <code>kill_agents</code> terminates them.
pause	Suspends the coordinator. No new agents are spawned, but running agents continue.
resume	Resumes the coordinator and triggers an immediate tick.
reconfigure	Updates <code>max_agents</code> , <code>executor</code> , <code>poll_interval</code> , or <code>model</code> at runtime without restart.
heartbeat	Records a heartbeat for an agent (used for liveness tracking).

The `reconfigure` command is particularly useful for live tuning. If a fan-out creates twenty parallel tasks and you only have five slots, you can bump `max_agents` to ten without stopping anything. When the fan-out completes and work converges, scale back down.

Observing the System

The IPC protocol lets tools talk to the daemon. But many integrations need to observe the graph from the outside—a CI system that triggers on task completion, a dashboard that tracks agent progress, a portfolio manager that records outcomes. For these, `workgraph` provides `wg watch`.

`wg watch` streams a real-time event feed of graph mutations to standard output. Each line is a JSON object with a type, timestamp, optional task ID, and a data payload carrying the operation detail. The event types mirror the operations log: `task.created`, `task.started`, `task.completed`, `task.failed`, `task.retried`, `evaluation.recorded`, `agent.spawned`, `agent.completed`. The stream reads from the same provenance log that records every mutation to the graph—`wg watch` is not a separate event system but a live tail of the log with structured formatting.

Events can be filtered. The `--event` flag accepts categories—`task_state` for all task transitions, `evaluation` for scoring events, `agent` for spawn and completion. The `--task` flag narrows to events affecting a specific task by ID prefix. These filters compose: you can watch only state-change events for tasks in a particular subtree. The `--replay N` flag emits the last `N` historical operations before switching to live streaming, letting a newly launched adapter catch up on recent history without scanning the full log.

The Adapter Pattern

`wg watch` is one side of a broader integration architecture. External systems interact with `workgraph` through five ingestion points, each corresponding to a different kind of information flow:

Point	Command	What flows
Evaluation	<code>wg evaluate record</code>	Scores with source tags — external outcome data enters the agency's performance records.
Task	<code>wg add</code>	New work items — an external system can inject tasks with dependencies, skills, and descriptions.

Point	Command	What flows
Context	<code>wg trace import</code>	Peer exports and knowledge artifacts — enriching agent prompts with cross-boundary data.
State	<code>wg done</code> , <code>wg fail</code> , <code>wg log</code>	Status changes and progress events — an external system can mark work complete or record observations.
Observation	<code>wg watch</code>	The event stream <i>out</i> — external systems observe what is happening without polling.

The generic adapter follows a four-step pattern: *observe* the graph via `wg watch`, *translate* external data into workgraph’s vocabulary, *ingest* via the appropriate CLI command, and *react* by triggering external actions. A CI adapter might observe `task.completed` events, run a test suite, and record the result via `wg evaluate record --source "ci:tests"`. A portfolio manager might observe agent completions, measure real-world outcomes, and feed scores back as external evaluations. The adapter pattern is deliberately simple—each integration is a small loop of observe, translate, ingest, react—because the ingestion points are stable CLI commands, not a bespoke API.

The Operations Log and Trace

Every mutation to the graph—task creation, status change, evaluation, agent spawn—is recorded in the operations log (`operations.jsonl`). This log is the raw material for both `wg watch` (live streaming) and `wg trace` (historical reconstruction). The coordinator does not maintain a separate event bus; `wg watch` simply tails the operations log and formats each entry as a typed JSON event.

The trace system builds on this foundation. `wg trace show` reconstructs the history of a task or subtree by reading the operations log and replaying state transitions. `wg trace show --animate` takes this further: it reconstructs temporal snapshots of the graph at each mutation, then plays them back in the terminal as an interactive animation—tasks transitioning between statuses over time, a visual record of how work flowed through the graph. You can pause, step forward and backward through snapshots, and adjust playback speed.

`wg trace export --visibility <zone>` produces a filtered, shareable snapshot of the trace. The visibility parameter controls what crosses organizational boundaries: `internal` exports everything, `public` sanitizes the export (task structure without agent output, logs, or evaluations), and `peer` provides richer detail for trusted peers (including evaluations with notes stripped). The corresponding `wg trace import` ingests a peer’s export, namespacing imported tasks to avoid ID collisions and tagging evaluations with their origin for provenance tracking. These exports use the visibility field on each task (see §2) to determine what is included at each zone level.

These capabilities—watch, trace, export, import—form a layered system. The operations log is the ground truth. The watch stream is its real-time face. The trace commands are its analytical tools. And the export/import mechanism is how organizational memory crosses boundaries.

Custom Executors

Executors are defined as TOML files in `.workgraph/executors/`. Each specifies a command, arguments, environment variables, a prompt template, a working directory, and an optional timeout. The default `claude` executor pipes a prompt file into the Claude CLI with `--print` and `--output-format stream-json`. The default `shell` executor runs a bash command from the task’s `exec` field.

Custom executors enable integration with any tool. An executor for a different LLM provider, a code execution sandbox, a notification system—any process that can be launched from a shell command can serve as an executor. The prompt template supports the same `{{task_id}}`, `{{task_title}}`,

{{task_description}}, {{task_context}}, and {{task_identity}} variables as the built-in executors.

The executor also determines whether an agent is AI or human. The `claude` executor means AI. Executors like `matrix` or `email` (for sending notifications to humans) mean human. This distinction matters for auto-evaluation: human-agent tasks are skipped.

Pause, Resume, and Manual Control

The coordinator can be paused via `wg service pause`. In the paused state, no new agents are spawned, but running agents continue their work. This is useful when you need to make manual graph edits without the coordinator racing to dispatch tasks you are still arranging.

`wg service resume` lifts the pause and triggers an immediate tick.

For debugging and testing, `wg service tick` runs a single coordinator tick without the daemon. This lets you step through the scheduling logic one tick at a time, observing what the coordinator would do. And `wg spawn <task-id> --executor claude` dispatches a single task manually, bypassing the daemon entirely.

The Full Picture

Here is what happens, end to end, when a human operator types `wg service start --max-agents 5` on a project with tasks and an agency:

The daemon forks into the background. It opens a Unix socket, reads `config.toml` for coordinator settings, and writes its PID to the state file. Its first tick runs immediately.

The tick reaps zombies (there are none yet), checks the agent registry (empty), and counts zero alive agents out of a maximum of five. If `auto_assign` is enabled, it scans for ready tasks without agent identities and creates assignment meta-tasks. If `auto_evaluate` is enabled, it creates evaluation tasks for work tasks. It saves the graph if modified, then finds ready tasks.

Suppose three tasks are ready: two assignment meta-tasks and one task that was already assigned. The coordinator spawns three agents (five slots available, three tasks ready). Each spawn follows the dispatch cycle: resolve executor, resolve model, build context, render prompt, write wrapper script, claim task, fork process, register agent.

The three agents run concurrently. The two assigners examine the agency roster and bind identities. They call `wg done assign-{task-id}`, which triggers `graph_changed` IPC. The daemon wakes for an immediate tick. Now the two originally-unassigned tasks are ready (their assignment predecessors are done). The coordinator spawns two more agents. All five slots are full.

Work proceeds. Agents call `wg log` to record progress, `wg artifact` to register output files, and `wg done` when finished. Each `wg done` triggers another tick. Completed tasks unblock their dependents. The coordinator spawns new agents as slots open. If an agent crashes, the next tick detects the dead PID, triages the output, and either marks the task done, injects recovery context and reopens it, or restarts it cleanly.

The graph drains. Tasks move from open through in-progress to done. Evaluation tasks score completed work. Eventually the coordinator finds no ready tasks and all tasks terminal. It logs: “All tasks complete.” The daemon continues running, waiting for new tasks. The operator adds more work with `wg add`, the `graph_changed` signal fires, and the cycle begins again.

This is coordination: a loop that converts a plan into action, one tick at a time.